

# Technical Reference

## TDS Toolkit

Writing a Protocol Decoder

Document TDSTK-DEC-01  
TDS500 / TDS600 / TDS700 Series

**TDSTK-DEC-01**

Copyright © 2026. All rights reserved.

This document describes the decoder plug-in interface of TDS Toolkit. Information in this publication supersedes that in all previously published material.

TDS Toolkit is host software; it is not a Tektronix product. Tektronix and TDS are used only to name the instruments the software supports.

# Table of Contents

|                                 |            |
|---------------------------------|------------|
| Related Manuals                 | iv         |
| <b>1 The Decoder Model</b>      | <b>1-1</b> |
| What a Decoder Is               | 1-1        |
| One File, One Protocol          | 1-1        |
| The Contract                    | 1-1        |
| Discovery                       | 1-1        |
| Where the Files Live            | 1-1        |
| <b>2 The Signal</b>             | <b>2-1</b> |
| What a Decode Is Given          | 2-1        |
| Samples and Timing              | 2-1        |
| The Threshold Is Set            | 2-1        |
| Levels, Not Samples             | 2-1        |
| Where a Signal Comes From       | 2-2        |
| <b>3 The Decoder Contract</b>   | <b>3-1</b> |
| Class Attributes                | 3-1        |
| The Four Attributes             | 3-1        |
| Roles and Sources               | 3-1        |
| The Methods                     | 3-1        |
| decode                          | 3-1        |
| Parameters                      | 3-2        |
| The Param Type                  | 3-2        |
| <b>4 Frames and Rendering</b>   | <b>4-1</b> |
| The Frame                       | 4-1        |
| One Decoded Event               | 4-1        |
| Status                          | 4-1        |
| Rendering the Result            | 4-2        |
| Four Small Methods              | 4-2        |
| Colour                          | 4-2        |
| <b>5 Constructing a Decoder</b> | <b>5-1</b> |
| A Worked Example                | 5-1        |

|                                  |            |
|----------------------------------|------------|
| The Protocol                     | 5-1        |
| The Steps                        | 5-1        |
| The Code                         | 5-1        |
| Imports and Attributes           | 5-1        |
| Recovering the Bits              | 5-2        |
| Gathering the Words              | 5-2        |
| Rendering                        | 5-3        |
| The Serial Shortcut              | 5-3        |
| Do Not Re-implement Bit Sampling | 5-3        |
| <b>6 Installing and Testing</b>  | <b>6-1</b> |
| Installing                       | 6-1        |
| Drop In and Open                 | 6-1        |
| Testing                          | 6-1        |
| Drive It With a Synthetic Signal | 6-1        |
| Test Every Decoder               | 6-1        |
| Checklist                        | 6-2        |
| Before Relying On It             | 6-2        |



## List of Tables

|                                           |     |
|-------------------------------------------|-----|
| Table 1–1: Files of the decoder subsystem | 1–2 |
| Table 2–1: Members of a Signal            | 2–2 |
| Table 3–1: Decoder class attributes       | 3–1 |
| Table 3–2: Decoder methods                | 3–2 |
| Table 3–3: Param kinds                    | 3–2 |
| Table 4–1: Frame fields                   | 4–1 |
| Table 4–2: Frame status bits              | 4–2 |

# Preface

This manual describes how to add a protocol decoder to TDS Toolkit. A decoder is one Python file. It reads a captured waveform and returns a list of decoded events; the program lists it, maps its inputs, runs it and draws the result without any further code.

The reader is assumed to be a working programmer comfortable with Python. No knowledge of the rest of the program is needed: the whole interface is one base class and three small data types, all in the module `tds_decode.py`.

Read this manual beside two files that ship with the program: `decoders/_template.py`, the skeleton to copy, and `decoders/README.txt`, the short form of what follows.

## Related Manuals

`decoders/_template.py`  
`decoders/README.txt`  
`tds_decode.py`





# The Decoder Model

## What a Decoder Is

### One File, One Protocol

A decoder is a single Python file in the decoders folder that defines one subclass of Decoder. Each file is one protocol. The Waveform tab lists every decoder it finds, so adding a protocol is one file and nothing else in the program changes.

A decode runs entirely on a captured waveform that the Waveform tab already holds: a channel read from the instrument, or a stored reference loaded from a file. Nothing in a decoder touches the instrument. That is the point of the design. It works on any TDS, including the base models whose firmware cannot run a bus-decode application of their own, because the decode happens in the host, on samples, after the capture.

### The Contract

The whole interface is the Decoder base class. A decoder sets four class attributes and implements one method. The other three methods have working defaults, overridden only to render the protocol's own fields in the table and on the graph. The three data types a decoder handles, Signal, Frame and Param, are described in the chapters that follow.

### Discovery

At start-up, and whenever the Waveform tab is opened, the program scans the decoders folder for \*.py files, imports each one in isolation, and instantiates every Decoder subclass it finds. A file whose name begins with an underscore is treated as a helper and skipped, which is how \_template.py stays a template and never appears as an empty protocol in the list.

Import is defensive. A file that will not import, whether from a syntax error or a bad reference, is skipped, with a note written to the program log, rather than taking the whole list down with it. A half-finished decoder never hides the working ones. When two folders offer a decoder of the same NAME, the later folder wins, so a locally written copy overrides the bundled one.

### Where the Files Live

The decoders that ship with the program live in a decoders folder inside the installation. A writable copy is kept beside the settings file, tdstoolkit.json, and is topped up from the bundled set every run, exactly like the mask library. A decoder added in a later build is copied across without overwriting a locally written one; a deleted bundled decoder returns on the next run, because the bundled set is meant to be present. Drop a new file into the writable folder and it appears in the list the next time the tab is opened. No rebuild is needed.

**Table 1-1: Files of the decoder subsystem**

| File                  | Role                                                                                                                                                                                    |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tds_decode.py         | The engine: the Decoder base class, the Signal, Frame and Param types, the status constants, the shared serial helper, and discovery. A decoder imports from it and does not change it. |
| decoders/_template.py | The skeleton. Copy it to start a new decoder.                                                                                                                                           |
| decoders/README.txt   | The short-form instructions and a map of the example decoders.                                                                                                                          |
| decoders/.py          | One protocol each: uart.py, i2c.py, spi.py, can.py and the rest. Read the one nearest to the protocol being added.                                                                      |

# The Signal

## What a Decode Is Given

### Samples and Timing

The `decode()` method is handed one `Signal` per input the protocol needs. A `Signal` is one captured record reduced to what a decoder wants: the integer samples, and the timing that turns a sample index into a time. It carries no volts and no display scaling. A decoder compares a sample against a threshold and never needs to know the voltage, so the raw digitizer counts are passed through unscaled and a quarter-million-point record costs nothing to hand over.

### The Threshold Is Set

The decision threshold, the hysteresis dead band and the signal span are computed before `decode()` is called. A decoder does not compute a threshold of its own. The threshold is the midpoint of the record's extremes; the hysteresis is an eighth of the span, wide enough to reject edge ringing without a slow edge failing to cross it. These values are read, not set.

### Levels, Not Samples

For most protocols the useful form of the record is not the raw samples but a clean logical level per sample. The `digitize()` method returns exactly that: a 0 or 1 for every sample, with the record's own hysteresis applied so that edge ringing does not read as several crossings. Call it once and walk the result. A protocol that needs the raw counts, a bipolar line with two thresholds for instance, reads `sig.data` directly instead.

**Table 2–1: Members of a Signal**

| Member               | Meaning                                                                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| sig.data             | The integer samples, an indexable sequence of raw digitizer counts.                                                                     |
| sig.n                | The number of samples, the same as len(sig.data).                                                                                       |
| sig.dt               | Seconds per sample.                                                                                                                     |
| sig.t0               | The time of the first sample, in seconds relative to the trigger.                                                                       |
| sig.first_index      | The record offset of the first sample. Add it to a local sample index for an absolute record position.                                  |
| sig.threshold        | The decision level, in counts. Set by the framework.                                                                                    |
| sig.hysteresis       | The dead band about the threshold, in counts. Set by the framework.                                                                     |
| sig.span             | The difference between the record's highest and lowest sample. Set by the framework; a span near zero means a flat, signal-free source. |
| sig.time_at(i)       | The time in seconds at local sample index i, that is $t0 + i \times dt$ .                                                               |
| sig.digitize()       | A new bytearray of one 0/1 level per sample, with hysteresis.                                                                           |
| sig.auto_threshold() | Recomputes threshold, hysteresis and span. Already called by the framework; called only on a Signal built by hand (see Section 6).      |

---

**NOTE.** A Signal carries raw counts, not volts. Compare against sig.threshold, never against a voltage. A decoder that reads correctly at one vertical scale then reads correctly at every other scale, because the threshold moves with the record.

---

### Where a Signal Comes From

The Signal that decode() receives is built by the program, not by the decoder, from the captured Waveform with the class method Signal.from\_waveform(). It reads the integer levels and the sample interval and trigger offset out of the waveform preamble, constructs the Signal, and calls auto\_threshold() on it before handing it to the decoder. A Signal is built by hand only when writing a test, which Section 6 covers.

# The Decoder Contract

## Class Attributes

### The Four Attributes

A decoder subclass sets four class attributes. `NAME` and `DESCRIPTION` name it in the protocol list. `SOURCES` declares the waveform inputs it needs. `PARAMS` declares the settings the tab offers. Everything else is inherited.

**Table 3-1: Decoder class attributes**

| Attribute                 | Meaning                                                                                                                                                                                  |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>NAME</code>         | The protocol name. Shown in the list and used as the title of the graph label. Leave it blank, or "?", and the loader does not list the class; that is how the template hides.           |
| <code>DESCRIPTION</code>  | One line for the second column of the list.                                                                                                                                              |
| <code>SOURCES</code>      | A list of role names, in order. One role is a one-wire protocol; two or more each get a captured source mapped to them. A trailing "?" marks an optional role that may be left unmapped. |
| <code>PARAMS</code>       | A list of Param objects, one control each. May be empty.                                                                                                                                 |
| <code>COLOUR</code>       | The chevron colour for a clean frame, as "#rrggbb". A decoder owns its colours; the app has no colour setting for decoders. Set it for one colour for the whole protocol.                |
| <code>FAULT_COLOUR</code> | The chevron colour for a faulted frame. Red by default, so an error stands out on any protocol's colour.                                                                                 |

### Roles and Sources

`SOURCES` is both the count of waveforms a protocol needs and their names. A one-wire protocol declares one role, by convention "Line". A two-wire protocol such as I2C declares ["SDA", "SCL"]; the tab shows one source selector per role, and `decode()` receives a Signal for each. A trailing question mark, as in SPI's ["CLK", "MOSI", "MISO?", "CS?"], marks a role the operator may leave unmapped; its Signal is then absent from the dictionary `decode()` receives, and the `decode()` code checks for it.

## The Methods

### `decode`

`decode(self, signals, params)` is the one required method. `signals` is a dictionary mapping each role name, without any trailing "?", to its Signal; an optional role that was not mapped is absent. `params` is a dictionary mapping each Param key to the value the operator chose. The method returns a two-tuple: a list of Frame objects, and a one-line note string.

The note is shown above the results table. Use it to report what the decode inferred, such as the detected bit rate, the samples per bit and the number of frames found, or, when the record cannot be read, to say why. On failure, return an empty frame list and a note that explains the problem in one phrase.

**Table 3–2: Decoder methods**

| Method                  | Returns                                                                                                                                                             |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| decode(signals, params) | (frames, note). The decode itself. Required.                                                                                                                        |
| columns()               | The table headings after the Time column, as a list of (heading, width_in_pixels) pairs. Override to name the protocol's fields.                                    |
| row(frame, ascii)       | The table cells for one frame, matching columns() after the time. Override to format the protocol's fields.                                                         |
| label(frame, ascii)     | The shortest useful string for the on-graph chevron. Override to keep it terse.                                                                                     |
| colour(frame)           | The chevron colour for one frame. Defaults to COLOUR for a clean frame and FAULT_COLOUR for a faulted one. Override to colour per chevron, switching on frame.kind. |

The ascii argument passed to row() and label() is the state of the tab's Hex / ASCII toggle. Honour it where it makes sense, a byte shown as a character or as two hex digits, and ignore it where it does not.

## Parameters

### The Param Type

Each Param in the PARAMS list becomes one control on the tab, so a protocol adds its own options without any user-interface code. The constructor is Param(key, label, kind, choices, default, lo, hi, unit, note). key is the dictionary key decode() reads; label is the caption the operator sees; kind is one of three strings.

**Table 3–3: Param kinds**

| kind     | Control and fields                                                                                                                                                |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "choice" | A drop-down. choices is a list of (value, label) pairs; the value is what reaches params, the label is what is shown. default is one of the values.               |
| "int"    | An integer entry. lo and hi bound it; unit is a caption suffix such as "bits". A default of 0 is by convention taken to mean "auto", which the note should state. |
| "bool"   | A checkbox. default is True or False.                                                                                                                             |

Where default is omitted it falls back to the first choice for a choice, False for a bool, and 0 for an int. The optional note is shown beside the control as a hint.



# Frames and Rendering

## The Frame

### One Decoded Event

A Frame is one decoded event: a UART character, an I2C byte, a SPI word, a bus condition. The type is flat and shared by every protocol. It carries a primary datum, an optional secondary, a length, a status and a kind. What the fields mean is the decoder's business, since only the decoder that made a frame renders it, but their shape is fixed.

Construct a Frame with keyword arguments; every field has a default. The two index fields matter most and are the easiest to get wrong. `index` and `end_index` are absolute record sample positions: the graph draws the chevron box from one to the other. Form them as `sig.first_index` plus the local sample index, exactly as the time comes from `sig.time_at()` of the local index.

**Table 4-1: Frame fields**

| Field                  | Meaning                                                                                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>time</code>      | The event's start time in seconds, from <code>sig.time_at(local_index)</code> .                                                                                               |
| <code>index</code>     | The absolute start sample: <code>sig.first_index</code> + local start.                                                                                                        |
| <code>end_index</code> | The absolute end sample: <code>sig.first_index</code> + local end. The box spans <code>index</code> to <code>end_index</code> .                                               |
| <code>value</code>     | The primary datum: the byte, the word, the measured value.                                                                                                                    |
| <code>tag</code>       | An optional secondary datum: an I2C address, a SPI MISO word, a measured frequency. 0 if unused.                                                                              |
| <code>length</code>    | The datum's length, in whatever unit the protocol counts. Defaults to 1.                                                                                                      |
| <code>status</code>    | A bitwise OR of the status constants; OK (zero) when the frame is clean.                                                                                                      |
| <code>kind</code>      | A free string, such as "byte", "addr", "data", "start" or "stop", that <code>row()</code> and <code>label()</code> switch on to render each part of a frame in its own right. |

### Status

`status` is a bitmask. A clean frame is OK, which is zero. A frame with trouble ORs together one or more of the constants in Table 4-2. The framework draws any frame whose status is non-zero as an error, a red chevron and a leading exclamation in the label, without the decoder doing anything beyond setting the bit.

**Table 4-2: Frame status bits**

| Constant      | Value | Meaning                                                                                      |
|---------------|-------|----------------------------------------------------------------------------------------------|
| OK            | 0     | Clean; no error.                                                                             |
| FRAMING_ERROR | 1     | The frame did not close as its format requires, such as a UART stop bit that was not a mark. |
| PARITY_ERROR  | 2     | A parity bit disagreed with the data.                                                        |
| TRUNCATED     | 4     | The record ended part-way through the frame.                                                 |
| ACK_ERROR     | 8     | A byte that expected an acknowledgement was not acknowledged.                                |

Import the constants used from `tds_decode` alongside `Decoder`, `Frame` and `Param`. Define no status values beyond these; the five above are what the renderer understands.

## Rendering the Result

### Four Small Methods

The base class renders a frame as a plain byte: hex, ASCII and a status word. Override `columns()`, `row()` and `label()` to speak the protocol's vocabulary instead. `columns()` returns the headings after the Time column, each with a pixel width. `row()` returns one cell per heading, in the same order. `label()` returns the shortest useful string for the chevron drawn on the waveform itself.

A protocol whose frames carry different kinds switches on `frame.kind` in these methods, so that a START, an address byte and a data byte each read as themselves in the table and on the graph. Keep `label()` terse: it is drawn inside a small box over the trace, so one to three characters is the right length.

### Colour

The fourth method, `colour(frame)`, sets the chevron colour, and the decoder owns it completely: the app has no colour setting for decoders. The default paints a clean frame in the class `COLOUR` and a faulted one in `FAULT_COLOUR`, so setting `COLOUR` gives the whole protocol one colour. Override `colour()` and switch on `frame.kind` to paint a colour per chevron, so one line's address, data and framing each read in a colour of their own. Return a `"#rrggbb"`; fall back to the base class for the kinds the decoder does not colour itself.

# Constructing a Decoder

## A Worked Example

### The Protocol

This chapter builds a complete decoder from the template. The example protocol is a single self-clocked wire that carries one bit per pulse: the line rises, stays high for a fraction of the bit period, and falls; a short high is a 0 and a long high is a 1, decided by whether the high time is more than half the period. Bits are gathered into words of a chosen size and bit order. This is the family of the WS2812 pixel line and the NEC infra-red remote, and it is not among the built-in protocols, so it is a genuine new decoder rather than a copy of one.

### The Steps

1. *Copy the template:* Copy `decoders/_template.py` to `decoders/pulsebits.py`.
2. *Name the protocol:* Rename the class and set `NAME`, `DESCRIPTION` and `SOURCES`.
3. *Declare the settings:* Fill `PARAMS` with the word size and bit order controls.
4. *Write the decode:* Recover the bits from the level track, then gather them into words.
5. *Render the fields:* Override `columns()`, `row()` and `label()` to show the word in hex and binary.
6. *Test it:* Drive the decoder with a synthetic `Signal` and assert the result, as Section 6 shows.

## The Code

### Imports and Attributes

The header imports only from `tds_decode`, names the class, and declares the two inputs the tab will offer: a word size and a bit order. `SOURCES` holds one role, so the tab shows a single source selector.

```
from tds_decode import Decoder, Frame, Param
from tds_decode import OK, TRUNCATED

class PulseBitsDecoder(Decoder):

    NAME = "Pulse-width bits"
    DESCRIPTION = "Self-clocked pulse-width line, one wire"

    SOURCES = ["Line"]
    PARAMS = [
        Param("bits", "Word size", "int", default=8,
              lo=1, hi=32, unit="bits"),
        Param("order", "Bit order", "choice",
              choices=[("msb", "MSB first"),
                       ("lsb", "LSB first")],
              default="msb"),
    ]
```

### Recovering the Bits

decode() takes the one Signal, guards against a flat source, and reduces the record to a 0/1 level per sample with digitize(). It then walks the level track. Each bit begins on a rising edge; the run of high samples is the pulse, the run of low samples that follows carries on to the next rising edge, and the two together are the bit period. The high time against the period decides the bit.

```
def decode(self, signals, params):
    sig = signals["Line"]
    if sig.span < 8:
        return [], "no signal on this source"
    level = sig.digitize()
    n = len(level)
    word_bits = max(1, int(params.get("bits") or 8))
    msb = params.get("order", "msb") == "msb"

    bits = [] # (value, start_index, end_index)
    i = 1
    while i < n:
        if level[i] and not level[i - 1]: # a rising edge
            start = i
            while i < n and level[i]: # the high pulse
                i += 1
            high = i - start
            while i < n and not level[i]: # low, to next rise
                i += 1
            period = i - start
            if period > 0:
                bit = 1 if high * 2 > period else 0
                bits.append((bit, start, i))
        else:
            i += 1
```

### Gathering the Words

The bits are then taken in groups of word\_bits, most or least significant first as the parameter says. A group that comes up short because the record ended is emitted anyway, with its status marked TRUNCATED so the renderer draws it as an error. Each Frame takes its time from the first bit and spans from the first bit's start to the last bit's end. The note reports the totals.

```

frames = []
for base in range(0, len(bits), word_bits):
    chunk = bits[base:base + word_bits]
    value = 0
    for pos, (bit, _s, _e) in enumerate(chunk):
        shift = (word_bits - 1 - pos) if msb else pos
        value |= bit << shift
    start_local = chunk[0][1]
    end_local = chunk[-1][2]
    ok = len(chunk) == word_bits
    frames.append(Frame(
        time=sig.time_at(start_local),
        index=sig.first_index + start_local,
        end_index=sig.first_index + end_local,
        value=value, length=len(chunk),
        status=OK if ok else TRUNCATED,
        kind="data"))
return frames, "%d bit(s), %d word(s)" % (
    len(bits), len(frames))

```

## Rendering

The three render methods show the word as hex and as binary of its own bit width, and mark a truncated word. `label()` keeps to two hex digits with a leading exclamation on an error, which is the convention the built-in decoders follow.

```

def columns(self):
    return [("Time", 92), ("Hex", 60),
            ("Binary", 80), ("Status", 70)]

def row(self, frame, ascii=True):
    return ["0x%02X" % frame.value,
            format(frame.value, "0%db" % frame.length),
            "OK" if frame.status == OK else "CUT"]

def label(self, frame, ascii=True):
    text = "%02X" % (frame.value & 0xFF)
    return ("!" + text) if frame.status else text

```

That file is a complete, listable decoder. Dropped into the decoders folder, it appears in the tab as “Pulse-width bits” with a word-size box and a bit-order drop-down, and decodes any source mapped to its one input.

## The Serial Shortcut Do Not Re-implement Bit Sampling

The worked example samples bits itself because its protocol is self-clocked and unlike anything built in. For anything in the asynchronous-serial family, such as RS-232, RS-422/485, MODBUS, MIDI, LIN, XBee and DMX512, do not re-implement bit sampling. Call the shared, corpus-tested helper `serial_bytes()`, which does auto-baud, centre-of-bit sampling, and parity and stop-bit checks the one proven way, and build the framing on the byte stream it returns.

```
from tds_decode import Decoder, Param
from tds_decode import serial_bytes, STANDARD_BAUDS

class MyUartDecoder(Decoder):

    NAME = "My UART"
    DESCRIPTION = "Async serial, one wire"
    SOURCES = ["Line"]
    PARAMS = [
        Param("baud", "Bit rate", "choice",
              choices=[(0, "Auto")]
              + [(b, str(b)) for b in STANDARD_BAUDS],
              default=0),
        Param("framing", "Framing", "choice",
              choices=[(f, f) for f in
                       ("8N1", "8E1", "8O1", "8N2")],
              default="8N1"),
    ]

    def decode(self, signals, params):
        sig = signals["Line"]
        frames, err, baud, spb = serial_bytes(
            sig, int(params.get("baud") or 0),
            params.get("framing", "8N1"), inverted=False)
        if err:
            return [], err
        return frames, "%d baud, %.1f smp/bit, %d frames" % (
            baud, spb, len(frames))
```

`serial_bytes(sig, baud, framing, inverted)` returns four values: the byte frames, an error string that is empty on success, the detected baud rate, and the samples per bit. A baud of 0 asks it to detect the rate. `inverted True` reads an idle-low, active-high electrical sense such as true RS-232. A higher-level protocol such as MODBUS or LIN calls `serial_bytes` for the raw bytes and then groups and checks them into its own frames; read `modbus.py` and `lin.py` for that pattern.

# Installing and Testing

## Installing

### Drop In and Open

Copy the finished file into the writable decoders folder beside tdstoolkit.json and open, or re-open, the Waveform tab. The decoder appears in the protocol list. There is no registration step and no rebuild. Map a captured source to each of its roles, set its parameters, and run it.

---

**NOTE.** *If the decoder does not appear, it did not import. Check the program log for a line naming the file and the exception. The loader skips a file that will not import rather than failing the whole list, so a broken decoder fails quietly.*

---

## Testing

### Drive It With a Synthetic Signal

A decoder is a pure function of its Signals and parameters, so it is tested without an instrument by building a Signal from a constructed sample array, calling decode(), and asserting the result. Build the samples so their meaning is known, wrap them in a Signal, call auto\_threshold(), the one step the framework normally does, and check the frames.

```
from tds_decode import Signal
from pulsebits import PulseBitsDecoder

def _bit(value, period=40):
    """One pulse-width bit: high for part of the period."""
    high = int(period * (0.66 if value else 0.33))
    return [30000] * high + [-30000] * (period - high)

def demo():
    stream = [-30000] * 20          # idle low lead-in
    for bit in (1, 0, 1, 1, 0, 0, 1, 0): # 0xB2, MSB first
        stream += _bit(bit)
    stream += [-30000] * 20          # idle low tail
    sig = Signal(stream, 1e-6)       # 1 us per sample
    sig.auto_threshold()             # framework does this
    frames, note = PulseBitsDecoder().decode(
        {"Line": sig}, {"bits": 8, "order": "msb"})
    assert len(frames) == 1, note
    assert frames[0].value == 0xB2, "%02X" % frames[0].value
    print("ok:", note)

if __name__ == "__main__":
    demo()
```

### Test Every Decoder

Every built-in decoder is checked this way. The RS-232 decoder is run against a corpus of real captures; every other decoder is checked against a synthetic vector in `tdstest.py`, each with a matching mutation bite in `bitecheck.py` that breaks the decoder on purpose and confirms the test notices. Add a case for a protocol that is added.



---

**CAUTION.** *A decode that is not tested is a decode that will drift. Add a synthetic vector for every decoder added, and a mutation bite that proves the vector is load-bearing, before relying on the result.*

---

## Checklist

### Before Relying On It

- The file is named in lower case with no spaces and does not begin with an underscore.
- The class sets `NAME`, `DESCRIPTION`, `SOURCES` and `PARAMS`, and `NAME` is not blank.
- `decode()` returns a (frames, note) tuple in every path, including the failure paths.
- Frame index and `end_index` are absolute: `sig.first_index` plus the local sample index.
- An unreadable record returns an empty list and an explaining note, never an exception.
- `columns()`, `row()` and `label()` agree on the number and order of the fields.
- A synthetic test drives the decoder and asserts the decoded values.